WEB DEVELOPMENT TECHNOLOGIES

A practical overview of HTML, CSS, JavaScript and PHP

Study notes for beginners

Structure • Styling • Interactivity • Server-side programming

Contents

- 1. Introduction to Web Development
- 2. HTML: Structure and Content
- 3. CSS: Design and Layout
- 4. JavaScript: Interactivity and Logic
- 5. PHP: Server-Side Programming
- 6. How the Technologies Work Together
- 7. Tools and Workflow
- 8. Security, Accessibility and Best Practices
- 9. Common Mistakes
- 10. Learning Roadmap and Revision Questions

1. Introduction to Web Development

Web development is the process of creating websites and web applications. A website may display information, accept user input, process data, communicate with a database, or provide services such as shopping, banking and dashboards.

Front end vs back end

- **Front end:** The part users see and interact with in a browser. HTML defines structure, CSS controls presentation, and JavaScript adds behaviour.
- **Back end:** Server-side code that handles requests, business rules, authentication, data processing and database access. PHP is one language that can run on the server.
- **Database:** Stores persistent information, such as user accounts, products, orders and posts. Common choices include MySQL, PostgreSQL and SQLite.

A simplified request cycle

- A user enters a URL or clicks a link.
- The browser sends an HTTP request to a server.
- The server may run PHP code and query a database.
- The server returns a response, often HTML, JSON, CSS, JavaScript or images.
- The browser displays the page and runs client-side JavaScript.

Key terms

- **Client:** Usually the browser or app making a request.
- **Server:** A computer or program that receives and responds to requests.
- **HTTP/HTTPS:** Communication protocols; HTTPS encrypts traffic using TLS.
- **URL:** An address identifying a resource.
- **API:** A defined way for software systems to communicate, often using HTTP and JSON.
- **Responsive design:** Layouts that adapt to different screen sizes.

2. HTML: Structure and Content

HTML stands for HyperText Markup Language. It describes the structure and meaning of webpage content. HTML is a markup language, not a general-purpose programming language.

Basic HTML document

```
&lt;!doctype html&gt;
&lt;html lang="en"&gt;
&lt;head&gt;
&lt;meta charset="UTF-8"&gt;
&lt;meta name="viewport" content="width=device-width, initial-scale=1.0"&gt;
&lt;title&gt;My First Page&lt;/title&gt;
&lt;/head&gt;
&lt;body&gt;
&lt;h1&gt;Welcome&lt;/h1&gt;
&lt;p&gt;This is my first webpage.&lt;/p&gt;
&lt;a href="about.html"&gt;About us&lt;/a&gt;
&lt;/body&gt;
&lt;/html&gt;
```

Important parts

- **<!doctype html>:** Requests standards mode.
- **<html>:** Root element; lang identifies the document language.
- **<head>:** Metadata, title and linked resources.
- **<meta charset>:** Sets character encoding, usually UTF-8.
- **Viewport meta tag:** Helps mobile browsers size the page correctly.
- **<body>:** Content rendered in the page.

Common HTML elements

- **Headings:** <h1> through <h6>; keep a logical hierarchy.
- **Text:** <p>, , , <br>.
- **Links:** <a href="...">.
- **Images:** <img src="photo.jpg" alt="Description">. Useful alt text supports accessibility.
- **Lists:** and <ol> with <li> items.
- **Semantic structure:** <header>, <nav>, <main>, <section>, <article>, <footer>.
- **Forms:** <form>, <label>, <input>, <select>, <textarea>, <button>. Associate labels with inputs.
- **Tables:** <table>, <tr>, <th>, <td> for tabular data, not page layout.

Attributes and semantics

Attributes add information to elements. Common examples include href, src, alt, id, class, type, name, value, required and disabled. Semantic elements describe the purpose of content, improving maintainability and accessibility.

HTML form example

```
&lt;form action="/register" method="post"&gt;
&lt;label for="email"&gt;Email address&lt;/label&gt;
&lt;input id="email" name="email" type="email" required&gt;
&lt;button type="submit"&gt;Register&lt;/button&gt;
&lt;/form&gt;
```

The browser can check whether a field resembles an email address, but the server must still validate submitted data.

HTML best practices

- Use semantic elements and logical headings.
- Give meaningful images useful alt text; use empty alt text for decorative images.
- Use labels for form controls and buttons for actions.
- Keep content structure in HTML and presentation in CSS.
- Test on mobile and desktop.

3. CSS: Design and Layout

CSS stands for Cascading Style Sheets. It controls appearance: colours, fonts, spacing, borders, positioning, responsive layouts and animations.

Ways to add CSS

- **External stylesheet:** Usually the best default. Link a separate .css file.
- **Internal stylesheet:** CSS inside a <style> element in the document head.
- **Inline style:** CSS in an element's style attribute; use sparingly.

```
&lt;link rel="stylesheet" href="styles.css"&gt;
```

CSS rule anatomy

```
p {
color: navy;
font-size: 16px;
line-height: 1.5;
}
```

The selector (p) chooses elements. Declarations are property-value pairs. The cascade, specificity, inheritance and source order influence which rules apply.

Selectors

- **Element:** p selects paragraphs.
- **Class:** .card selects elements with class="card".
- **ID:** #header selects the element with id="header"; IDs should be unique.
- **Descendant:** nav a selects links inside nav.
- **Pseudo-class:** a:hover applies on hover; input:focus applies when focused.

The box model

Elements have content, padding, border and margin. Padding is inside the border; margin creates space outside it. A common sizing rule is:

```
*,
*::before,
*::after {
box-sizing: border-box;
}
```

Layout tools

- **Flexbox:** Arranges items along one main axis, such as a navigation bar.
- **CSS Grid:** Builds two-dimensional row-and-column layouts.
- **Positioning:** static, relative, absolute, fixed and sticky affect placement.
- **Media queries:** Apply styles at different screen widths or according to user/device preferences.

Flexbox example

```
.container {
display: flex;
gap: 1rem;
justify-content: space-between;
align-items: center;
flex-wrap: wrap;
}
```

Responsive design example

```
.cards {
display: grid;
grid-template-columns: repeat(3, minmax(0, 1fr));
gap: 1rem;
}
@media (max-width: 700px) {
.cards { grid-template-columns: 1fr; }
}
```

CSS best practices

- Use external stylesheets and meaningful class names.
- Prefer Flexbox and Grid over brittle positioning hacks.
- Test narrow screens, keyboard focus and zoomed text.
- Avoid excessive !important declarations.
- Remove unused rules and keep selectors understandable.

4. JavaScript: Interactivity and Logic

JavaScript is a programming language used in browsers and also on servers through runtimes such as Node.js. In browsers, it responds to clicks, validates forms, updates content and requests API data.

Variables and constants

```
const appName = "Study Portal";
let count = 0;
count = count + 1;
```

Use const by default, and let when reassignment is needed. Avoid var in most modern code because its scoping rules can be confusing.

Data types

- Primitive values include string, number, boolean, undefined, null, bigint and symbol.
- Objects include arrays, ordinary objects, dates and other structured values.

```
const username = "Ada";
const age = 21;
const isStudent = true;
const subjects = ["HTML", "CSS", "JavaScript"];
const user = { name: "Ada", active: true };
```

Operators and comparisons

JavaScript supports arithmetic (+, -, *, /), assignment, logical operators (&&, ||, !) and comparisons. Prefer strict equality (=== and !==) unless type coercion is intentional.

Conditions, loops and functions

```
if (age >= 18) {
console.log("Adult");
} else {
console.log("Under 18");
}
for (const subject of subjects) {
console.log(subject);
}
function add(a, b) { return a + b; }
const multiply = (a, b) => a * b;
```

The DOM and events

The Document Object Model represents the page as objects JavaScript can inspect and change. Event listeners let code respond to clicks, key presses, form submissions and other events.

```
const heading = document.querySelector("#title");
const button = document.querySelector("#change");
button.addEventListener("click", () => {
heading.textContent = "You clicked the button!";
});
```

Arrays and objects

```
const prices = [10, 20, 30];
const doubled = prices.map(price => price * 2);
const affordable = prices.filter(price => price < 25);
const product = { name: "Notebook", price: 12 };
console.log(product.name);
```

Asynchronous JavaScript and APIs

Network requests take time. Promises and async/await let JavaScript handle them without freezing the page. fetch() can request JSON from an API.

```
async function loadUsers() {
try {
const response = await fetch("/api/users");
if (!response.ok) throw new Error("Request failed");
const users = await response.json();
console.log(users);
} catch (error) {
console.error("Could not load users:", error);
}
}
```

HTTP errors such as 404 or 500 do not automatically reject fetch(); check response.ok. Production code should handle loading, empty and error states.

JavaScript best practices

- Do not treat client-side validation as security; validate on the server too.
- Avoid inserting untrusted input with innerHTML; use textContent for plain text.
- Never put secret API keys, passwords or private tokens in browser code.
- Handle network failures, unexpected data and HTTP errors.
- Use meaningful names, small functions and consistent formatting.

5. PHP: Server-Side Programming

PHP is a server-side scripting language used to generate web pages, process form submissions, build APIs and interact with databases. PHP executes on the server; the browser receives the resulting response, not the PHP source.

A first PHP example

```
&lt;?php
$message = "Hello from PHP";
echo $message;
?&gt;
```

Variables begin with $. Output can be produced with echo. Statements usually end with semicolons.

Variables and types

```
&lt;?php
$name = "Ada";
$age = 21;
$isMember = true;
$skills = ["HTML", "CSS", "PHP"];
echo htmlspecialchars($name, ENT_QUOTES, 'UTF-8');
?&gt;
```

PHP is dynamically typed. Common types include strings, integers, floats, booleans, arrays, objects and null. htmlspecialchars helps escape text when inserting it into HTML.

Conditions, loops and functions

```
&lt;?php
$score = 75;
if ($score >= 50) {
echo "Pass";
} else {
echo "Try again";
}
function total($a, $b) { return $a + $b; }
echo total(4, 6);
?&gt;
```

Handling a form

```
&lt;form method="post" action="process.php"&gt;
&lt;label for="name"&gt;Name&lt;/label&gt;
&lt;input id="name" name="name" required&gt;
&lt;button type="submit"&gt;Submit&lt;/button&gt;
&lt;/form&gt;

&lt;?php
if ($_SERVER["REQUEST_METHOD"] === "POST") {
$name = trim($_POST["name"] ?? "");
if ($name === "") {
http_response_code(400);
exit("Name is required.");
}
echo "Hello, " . htmlspecialchars($name, ENT_QUOTES, "UTF-8");
}
?&gt;
```

Never assume submitted data is valid. Validate meaning and format on the server and escape output for its context.

PHP and databases

PHP can connect to MySQL, PostgreSQL and other databases. PDO provides a consistent database interface. Use prepared statements rather than joining user input into SQL strings.

```
&lt;?php
$pdo = new PDO(
"mysql:host=localhost;dbname=school;charset=utf8mb4",
$dbUser,
$dbPassword,
[PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
);
$stmt = $pdo->prepare("SELECT id, name FROM students WHERE email = :email");
$stmt->execute(["email" => $email]);
$student = $stmt->fetch(PDO::FETCH_ASSOC);
?&gt;
```

The credentials are placeholders. Store real secrets in environment variables or protected configuration, not public source code.

Sessions and authentication

Sessions preserve server-side state across HTTP requests. A login system should use password_hash() and password_verify(), regenerate the session ID after login, configure secure cookies, protect state-changing requests against CSRF and enforce authorization on the server.

Modern PHP

- Frameworks such as Laravel and Symfony provide routing, validation, database tools, authentication features and testing support.
- PHP can render HTML on the server or expose JSON APIs for JavaScript front ends.
- Composer manages PHP packages and dependencies.

PHP best practices

- Validate input on the server and escape output appropriately.
- Use prepared SQL statements to reduce SQL injection risk.
- Never store passwords in plain text.
- Keep dependencies updated and do not expose detailed errors in production.
- Use HTTPS, least-privilege database accounts and protected secrets.

6. How the Technologies Work Together

Example: a contact form

- HTML creates form fields and a submit button.
- CSS styles the form and adapts it to mobile screens.
- JavaScript can check required fields, show loading feedback and send the form asynchronously.
- PHP receives the request, validates data, applies business rules and may save it to a database.
- The server returns a success or error response; JavaScript can update the page.

Simplified architecture

Browser: HTML + CSS + JavaScript → HTTPS request → PHP application → Database. The response returns to the browser. Real systems may also use caches, queues, storage services and external APIs.

Who does what?

Technology	Main responsibility	Runs mainly where?
HTML	Structure and meaning	Browser
CSS	Appearance and layout	Browser
JavaScript	Interaction, logic and API calls	Browser; also server via Node.js
PHP	Server logic, validation and database access	Server

HTML, CSS and JavaScript can be delivered to the browser, where users can inspect them. PHP executes on the server. Never rely on browser code to protect secrets or enforce permissions.

7. Tools and Development Workflow

- **Code editor:** Visual Studio Code or another editor.
- **Browser developer tools:** Inspect HTML/CSS, debug JavaScript, inspect network requests and view console errors.
- **Local PHP environment:** Install PHP and a web server, or use a local development stack. The built-in PHP server is useful for learning, not a complete production hosting setup.
- **Version control:** Git tracks changes; GitHub or another service stores remote repositories.
- **Package managers:** Composer manages PHP packages; npm manages JavaScript packages.
- **Testing:** Check validation, mobile layout, keyboard navigation, API errors, permissions and security.

A sensible workflow

- Plan the page and the data it needs.
- Write semantic HTML first.
- Add CSS for layout and responsive behaviour.
- Add JavaScript for interactions and API calls.
- Implement server-side PHP logic and database access.
- Test success cases, invalid input, permissions and failures.
- Use Git commits, deploy to a test environment, then deploy securely.

8. Security, Accessibility and Best Practices

Security checklist

- Use HTTPS for real applications.
- Validate input on the server; validation is not the same as escaping output.
- Use prepared statements for database queries.
- Escape output based on context. For HTML text, PHP htmlspecialchars() is commonly used.
- Use password_hash() and password_verify() for password handling.
- Check authentication and authorization on every protected server-side action.
- Protect state-changing requests from CSRF and configure cookies appropriately.
- Keep secrets out of public repositories and browser JavaScript.
- Update dependencies and avoid showing stack traces or credentials to users.

Accessibility checklist

- Use semantic HTML and meaningful page titles.
- Make important interactions usable with a keyboard.
- Give form controls visible, associated labels.
- Use descriptive link text and suitable image alternative text.
- Ensure readable contrast; do not convey meaning through colour alone.
- Show errors in text and make focus behaviour understandable.

Performance checklist

- Compress and size images appropriately.
- Avoid unnecessary JavaScript and oversized dependencies.
- Use caching thoughtfully and minimize repeated database work.
- Load only the resources needed and monitor performance.
- Test on slower devices and connections, not only powerful computers.

9. Common Beginner Mistakes

- Using HTML elements only for visual appearance instead of meaning.
- Fixing every layout issue with absolute positioning instead of learning Flexbox and Grid.
- Writing JavaScript that assumes elements exist or API requests always succeed.
- Assuming client-side validation prevents malicious requests.
- Building SQL strings from user input.
- Printing untrusted input into HTML without escaping.
- Storing passwords in plain text or putting API secrets in frontend files.
- Putting all code into one huge file instead of separating concerns.
- Ignoring mobile layout, keyboard access and error messages.

10. Learning Roadmap and Revision Questions

Suggested learning order

- **Stage 1, HTML:** Build a profile page with headings, links, images, lists and a contact form.
- **Stage 2, CSS:** Style the page and learn the box model, Flexbox, Grid, typography and responsive design.
- **Stage 3, JavaScript:** Learn variables, functions, arrays, objects, conditions, loops, DOM events, fetch and async/await.
- **Stage 4, PHP:** Learn syntax, request handling, validation, sessions, error handling and PDO database access.
- **Stage 5, SQL:** Learn tables, keys, relationships, SELECT, INSERT, UPDATE, DELETE, joins and transactions.
- **Stage 6, projects:** Build a to-do list, calculator, contact form, blog, login system and CRUD application.
- **Stage 7, professional practice:** Use Git, write tests, secure secrets, deploy to a test server and document the project.

Project ideas

- **Portfolio:** HTML/CSS, then JavaScript for interactive sections.
- **Expense tracker:** Forms, JavaScript validation, PHP API and database.
- **Blog:** Posts, categories, database CRUD, login and admin permissions.
- **Mini e-commerce site:** Product catalogue, cart and order creation. Verify prices on the server; never trust browser-submitted prices.
- **Student registration system:** CRUD records with validation and database constraints.

Revision questions

- 1. What is the difference between front end and back end?
- 2. Why is HTML called a markup language?
- 3. What is the CSS box model?
- 4. When would you use Flexbox rather than Grid?
- 5. What is the DOM, and what does addEventListener do?
- 6. Why should fetch code check response.ok?
- 7. Where does PHP execute, and what does the browser receive?
- 8. Why are prepared statements safer than concatenating SQL strings?
- 9. Why is server-side validation necessary even when HTML uses required?
- 10. Explain how all four technologies can work together.

Quick summary

HTML = structure. CSS = presentation. JavaScript = interactivity and logic. PHP = server-side logic. Together, these technologies can power everything from an informational site to a database-backed web application.

Official learning resources

Use MDN Web Docs for HTML, CSS and JavaScript, and the PHP Manual for PHP. Technology details evolve, so check current documentation when building real projects.

MDN Web Docs: developer.mozilla.org | PHP Manual: php.net/manual